

Do wykonania ćwiczeń należy uruchomić w terminalu symulację turtlesim:

- `roslaunch turtlesim turtlesim_node`

Zadania - Terminal

Zadanie 1

Dla podanych serwisów wyświetlić informację o typie wiadomości serwisowej. Podać nazwę paczki i typu. Wyświetlić strukturę wiadomości serwisowej.

`/clear` - wyczyszczenie narysowanych ścieżek

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:  
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

`/kill` - usunięcie robota

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:  
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

`/reset` - reset środowiska do stanu początkowego

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:  
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

`/spawn` - utworzenie nowego robota

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:  
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

`/turtle1/set_pen` - ustawienie koloru pędzla do rysowania

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:
```

```
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

/turtle1/teleport_absolute - natychmiastowe przeniesienie robota do wskazanej lokalizacji

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:  
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

/turtle1/teleport_relative - przeniesienie robota, współrzędne podawane w układzie robota

```
In [ ]: # Wyświetlenie informacji o typie wiadomości serwisowej.  
! Do uzupełnienia komenda w terminalu
```

```
In [ ]: # Nazwa paczki:  
# Nazwa typu wiadomości serwisowej:
```

```
In [ ]: # Wyświetlenie struktury wiadomości serwisowej  
! Do uzupełnienia komenda w terminalu
```

Zadanie 2

Utworzyć dodatkowe 2 roboty o różnych nazwach w różnych miejscach

```
In [ ]:
```

Zadanie 3

Narysować dowolnym robotem kwadrat z użyciem serwisów.

```
In [ ]:
```

Zadanie 4

Usunąć wszystkie roboty.

```
In [ ]:
```

Python - Klient

Dla podanych nazw serwisów należy utworzyć odpowiedniego klienta i wiadomość serwisową (żądanie) wysyłane przez klienta na serwer. Do realizacji zadania można wykonać następujące kroki:

1. Wykorzystać odczytany typ wiadomości serwisowej i strukturę z poprzednich zadań lub ponownie je wyświetlić
2. Zaimportować odpowiedni typ wiadomości serwisowych
3. Utworzyć obiekt klienta (ServiceProxy)
4. Utworzyć obiekt wiadomości serwisowej - żądanie klienta
5. Uzupełnić pola w wiadomości, jeśli trzeba
6. Wysłać żądanie

```
In [ ]: import rospy  
rospy.init_node("serwis_zadania")
```

/kill

Utworzyć klienta usuwającego żółwia turtle1

In []:

/reset

In []:

/turtle1/set_pen

Utworzyć klienta dla żółwia o nazwie turtle1 ustawiającego kolor pędzla na czerwony

In []:

/turtle1/teleport_absolute

Utworzyć klienta teleportującego absolutnie żółwia w dowolne położenie.

In []:

/turtle1/teleport_relative

Utworzyć klienta teleportującego względnie żółwia na podstawie aktualnej prędkości postępowej i obrotowej.

In []:

Python - Serwer

Dla podanych typów wiadomości serwisowych i nazw serwisów utworzyć serwer, który odpowiada na żądanie klienta. Do realizacji zadania można wykonać następujące kroki:

1. Wyświetlić strukturę wiadomości serwisowej
2. Zaimportować odpowiedni typ wiadomości serwisowych
3. Utworzyć obiekt serwera (Service)
4. Utworzyć obiekt wiadomości serwisowej - odpowiedź
5. Uzupełnić pola w wiadomości, jeśli trzeba
6. Wysłać odpowiedź

std_srvs/Empty

Utworzyć serwer o nazwie czyszczenie_mapy (działa tak samo jak /clear, ale jest to napisany własny serwer)

In []:

```
# wyświetlić strukturę wiadomości serwisowej
```

In []:

```
# utworzyć serwer
```

In []:

```
# Klient do uruchomienia i przetestowania działania serwera
# przed uruchomieniem należy cokolwiek narysować na mapie
empty_client = rospy.ServiceProxy("czyszczenie_mapy", Empty)
empty_client()
```

std_srvs/SetBool

-> utworzyć serwer o nazwie "reset_mapy" resetujący mapę wykorzystujący istniejący serwer reset. Dla wartości True mapa jest restartowana, a dla False nie.

In []:

```
# wyświetlić strukturę wiadomości serwisowej
```

In []:

```
# utworzyć serwer
```

```
In [ ]: # Klient do uruchomienia i przetestowania działania serwera
# przed uruchomieniem należy wykonać dowolne akcje zmieniające widok symulacji
from std_srvs.srv import SetBool, SetBoolRequest
map_reset_client = rospy.ServiceProxy("reset_mapy", SetBool)
```

```
In [ ]: # żądanie wymuszające wyczyszczenie symulacji
req = SetBoolRequest()
req.data = True
map_reset_client(req)
```

```
In [ ]: # żądanie nie powoduje wyczyszczenia symulacji
req = SetBoolRequest()
req.data = False
map_reset_client(req)
```

std_srvs/Trigger

Utworzyć serwer o nazwie "/turtle1/triangle_trigger". Po wyzwoleniu trigera robot ma narysować trójkąt.

```
In [ ]: # wyświetlić strukturę wiadomości serwisowej
```

```
In [ ]: # utworzyć serwer
```

```
In [ ]: # Klient do uruchomienia i przetestowania działania serwera
from std_srvs.srv import Trigger
trigger_client = rospy.ServiceProxy("/turtle1/triangle_trigger", Trigger)
trigger_client()
```